

Dynamic programming

atlantis

Date _____

Page _____

Exo

Fibonacci series -

$$f(n) = f(n-1) + f(n-2)$$

$$= 1 ; n=1$$

$$= 0 ; n=0$$

$$n=0 \quad 1 \quad 2 \quad 3$$

$$f(n)=0 \quad 1 \quad 3 \quad 5 \dots$$

-f(n)

{

if (n==0)

return 0;

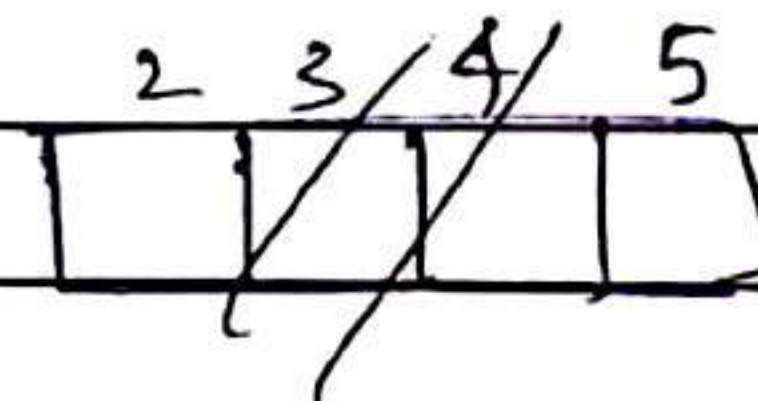
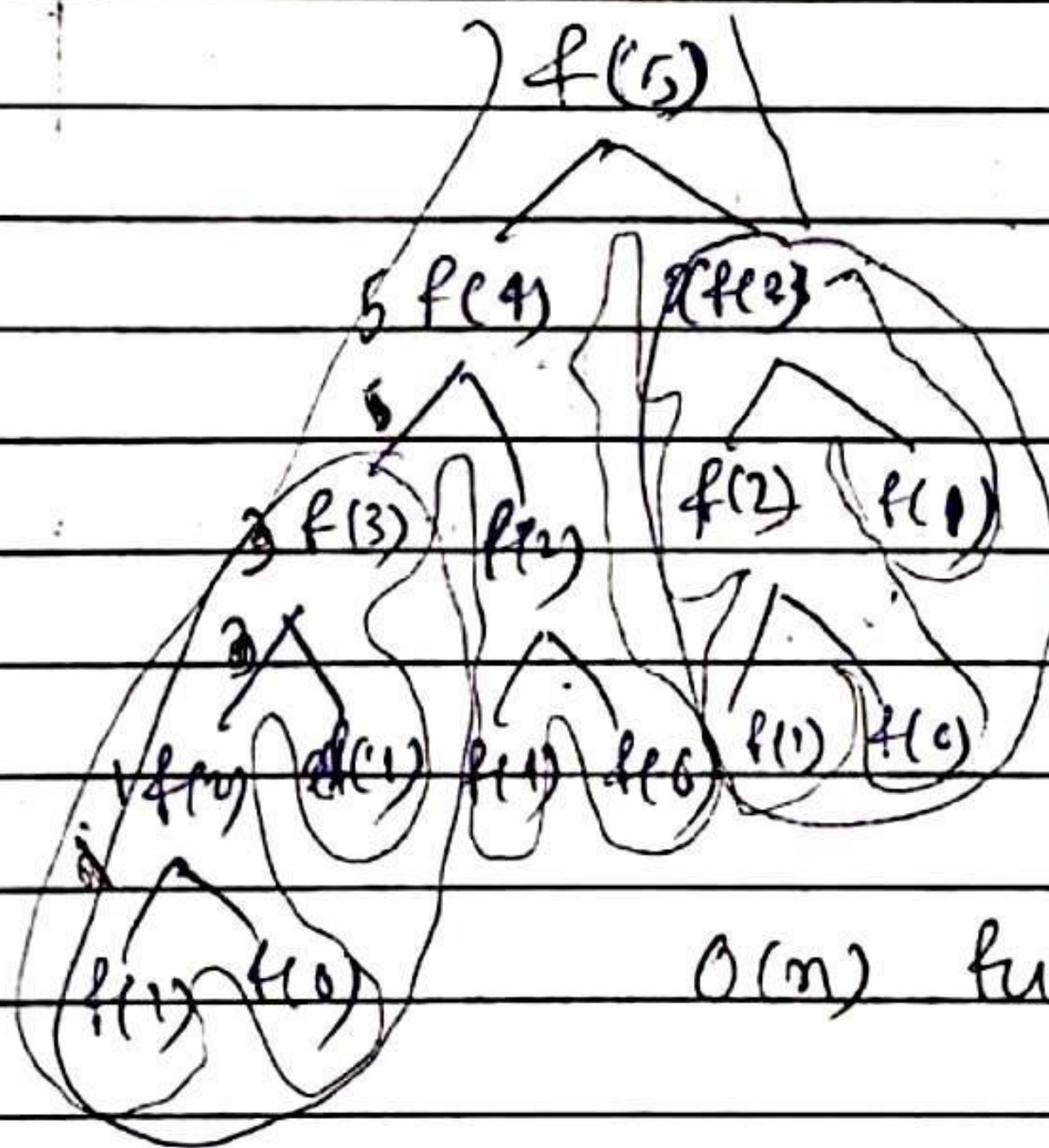
if (n==1)

return 1;

if

return (f(n-1) + f(n-2));

}



$O(n)$ function can make.

Check the table

for T(n)

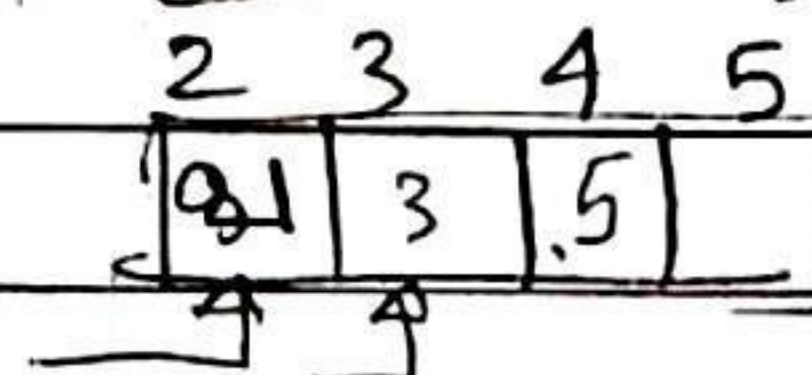
if table empty if T(n) is empty then we call function f(n)

for that if T(n) is empty

it is called dynamic programming.

$$T(n) = f(n)$$

Tabulization: saving the value in table and then using them later.



Matrix chain Multiplication

atlantis

Date _____

Page _____

• Matrix chain multiplication - Dynamic programming =

• Matrix multiplication =

$$\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \times \begin{bmatrix} 2 & 1 \\ 3 & 1 \end{bmatrix}$$

$$A(3 \times 2) = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}$$

$$= \begin{bmatrix} 4+3 & 2+1 \\ 2+6 & 1+2 \end{bmatrix}$$

$$= \begin{bmatrix} 7 & 3 \\ 8 & 3 \end{bmatrix}$$

$$B(2 \times 3) = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{bmatrix}$$

→ multiply to matrix see that they are compatible.
(no. of Column of first matrix == no. of Row of second-matrix)

here, ^{size} resulting matrix will be (3x3).

$$C = \begin{bmatrix} & & \\ & & \\ & & \end{bmatrix}_{3 \times 3}$$

two matrix - $\begin{bmatrix} & & \end{bmatrix}_{p \times q} \times \begin{bmatrix} & & \end{bmatrix}_{q \times r}$

$$\text{total no. of multiplication required} = 2 \times \boxed{p \times q \times r}$$

• Chain multiplication =

3-matrix A, B, C.

$$(A_{2 \times 1} B_{1 \times 2}) C_{2 \times 9}$$

$$A_{2 \times 1} (B_{1 \times 2} C_{2 \times 9})$$

both the case result will be same.

~~total multi required = 2x1x2 = 2~~

$$\begin{aligned} \textcircled{1} & ((A_{2 \times 1} \times B_{1 \times 2}) \times C_{2 \times 1}) \\ &= (D_{2 \times 2} \times C_{2 \times 1}) \quad \text{total multi required} = 2 \times 2 \times 1 = 4 \\ &= E_{2 \times 1} \end{aligned}$$

SO, total multiplication required = 4 + 16

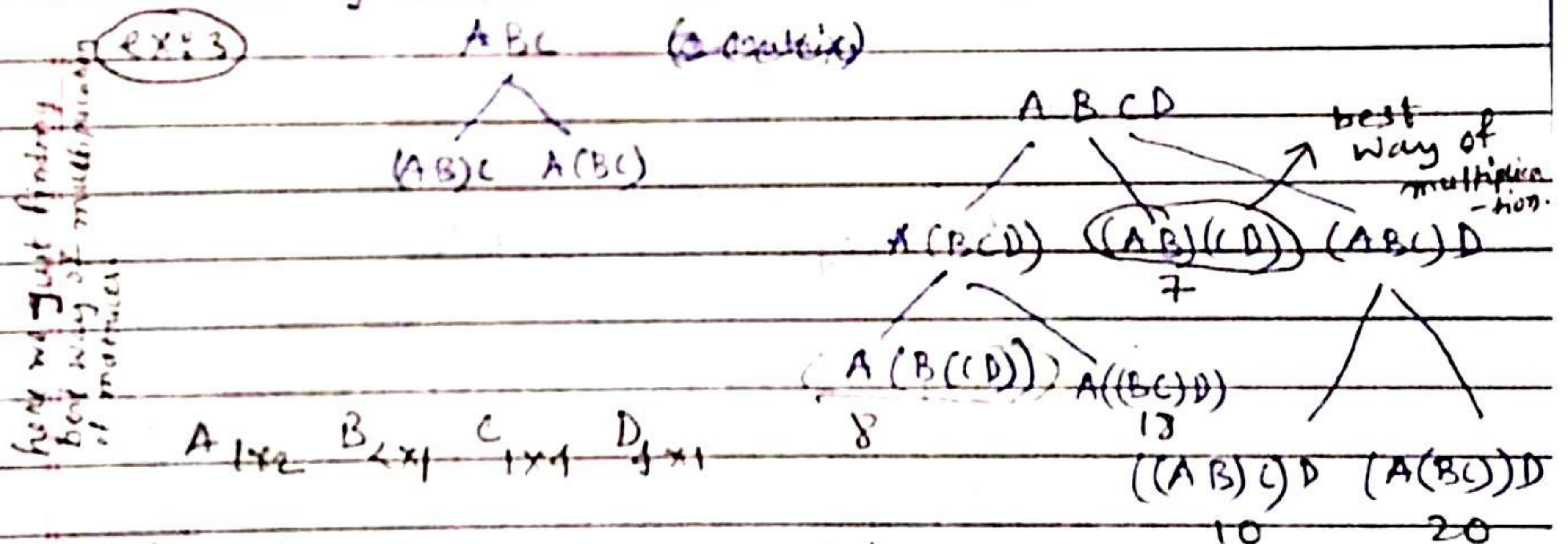
$$\textcircled{2} (A_{2 \times 1} (B_{1 \times 2} \times C_{2 \times 1})) \quad \text{multi req} = 1 \times 2 \times 1 = 2$$

$$= (A_{2 \times 1} \times D_{1 \times 1}) \quad \text{multi req} = 2 \times 1 \times 1 = 2$$

$$= E_{2 \times 1}$$

total multiplication req = 2 + 8 = 10

Way of parenthesis — A, B, C, D are four matrix



$$\textcircled{1} A(B(CD)) \quad \text{multi 1}$$

$$= A_{1 \times 2} (B_{2 \times 1} (C_{1 \times 1} \times D_{1 \times 1}))$$

$$= A_{1 \times 2} (B_{2 \times 1} D_{1 \times 1}) \quad \text{multi 2}$$

$$= (A_{1 \times 2} \times E_{2 \times 1}) \quad \text{multi 2}$$

$$= F_{1 \times 1}$$

total multi required = 4 + 2 + 2

= 8 ✓

$$(i) A(BC)D$$

$$= A_{1 \times 2} (B_{2 \times 1} C_{1 \times 4}) D_{4 \times 1}$$

$$8+8+2$$

$$= A_{1 \times 2} (E_{2 \times 4} D_{4 \times 1})$$

$$= (A_{1 \times 2} F_{2 \times 1})$$

$$= G_{1 \times 1}$$

in this case
so, total multiplication required = $8+8+2$
 $= 18$

$$(ii) (AB)(CD)$$

$$= (A_{1 \times 2} B_{2 \times 1}) (C_{1 \times 4} D_{4 \times 1})$$

$$\text{total multi req} = 4+2+1$$

$$= 7$$

$$= (E_{1 \times 1} F_{1 \times 1})$$

$$(iv) (AB)C)D$$

$$= (AB) ((A_{1 \times 2} B_{2 \times 1}) C_{1 \times 4}) D_{4 \times 1}$$

$$= (E_{1 \times 1} \times C_{1 \times 4}) D_{4 \times 1}$$

$$= (F_{1 \times 4} \times D_{4 \times 1})$$

$$\text{total multi req} = 2+4+4$$

$$= 10$$

$$(v) (A(BC))D$$

$$m = 2 \times 1 \times 4 = 8$$

$$8 + 8 + 1$$

$$= (20)$$

$$= (A_{1 \times 2} (B_{2 \times 1} C_{1 \times 4})) D_{4 \times 1}$$

$$= (A_{1 \times 2} BC_{2 \times 4})_{m=8} D_{4 \times 1}$$

$$= ((ABC)_{1 \times 4} \times D_{4 \times 1})_{m=1}$$

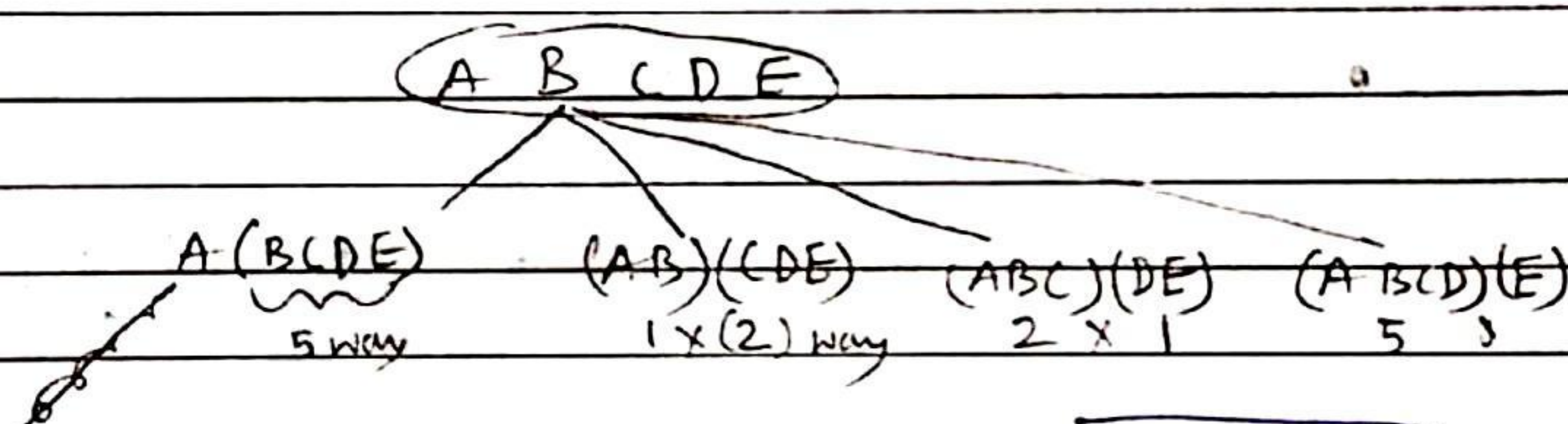
$$= (ABCD)_{1 \times 1}$$

$$\text{total multi req} = 8 + 8 + 4 = (20)$$

✓ If 3 matrix = 2 ways (of paranthesis)

$$4 \text{ " } = 5 \text{ "}$$

$$5 \text{ " } = 14 \text{ "}$$



$$\text{no. of way can paranthesis is} = \frac{(2n)!}{(n+1)! n!}$$

if multiply '4'-matrix, then
put $n=3$

if multi '5'-matrix, then
put $n=4$

$$= (2 \times 3)! \quad (n=3)$$

$$4! \cdot 3!$$

$$= \frac{6 \times 5}{6}$$

$$= 5$$

• Optimal Substructure and Recursive equations -

• Optimal Substructure =

multiply n matrices - $A_1 A_2 A_3 \dots A_n$

Dimension - $p_0 \times p_1 \quad p_1 \times p_2 \quad p_2 \times p_3 \dots p_{n-1} \times p_n$

$$A_i^0 \quad (p_{i-1} \times p_i)$$

$$\begin{array}{c} \cancel{A_i^0} + \cancel{A_{i+1}^0} + \cancel{A_{i+2}^0} \dots \\ A_i^0 \mid A_{i+1}^0 \mid A_{i+2}^0 \mid \dots \mid A_j^0 = (A_i^0 A_{i+1}^0 \dots A_k^0) \mid (A_{k+1}^0 \dots A_j^0) + \end{array}$$

$(p_{i-1} \times p_k) \qquad (p_k \times p_j)$

$$i^0 \leq k \leq j^0$$

$$\text{total mult req} = p_{i-1} p_k p_j$$

• Recursive equation =

$$m[i, j] = \begin{cases} 0 & i=j \\ \min \left\{ \underbrace{m[i, k]}_{(p_{i-1} \times p_k)} + \underbrace{m[k+1, j]}_{(p_k \times p_j)} + p_{i-1} p_k p_j \right\} \end{cases}$$

$$A_i^0 \quad A_{i+1}^0 \dots A_j^0$$

• Recursive tree =

4-matrix: $A_1 \mid A_2 \mid A_3 \mid A_4$

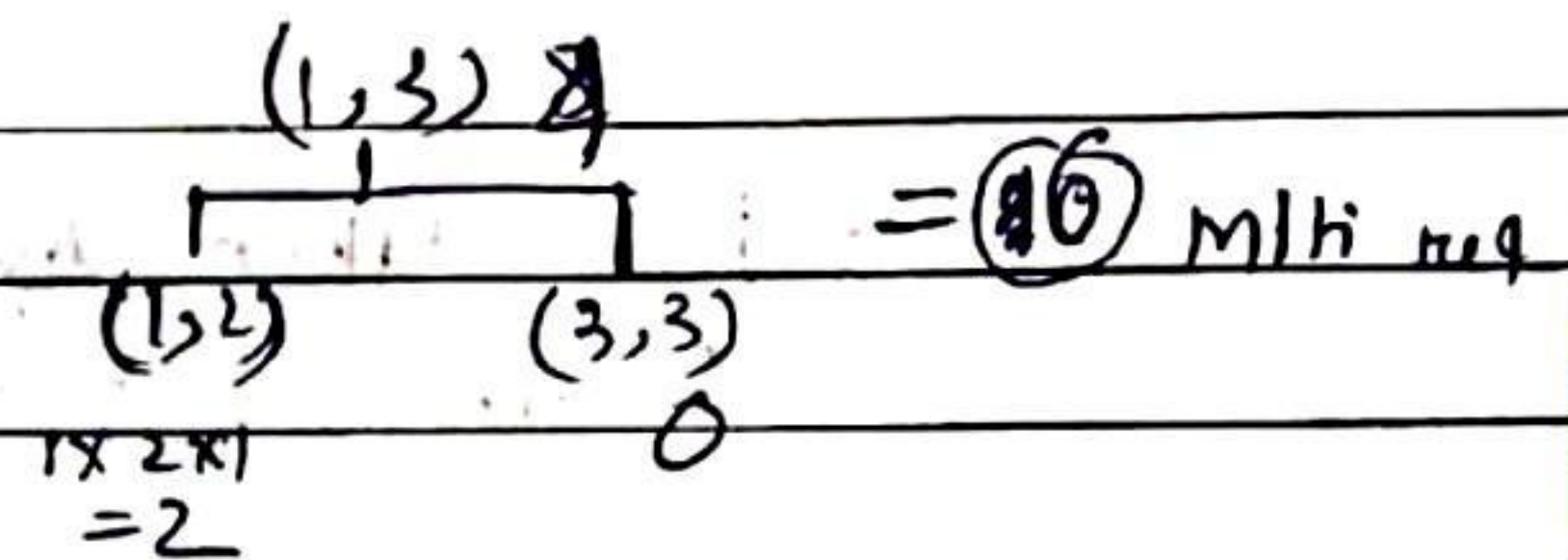
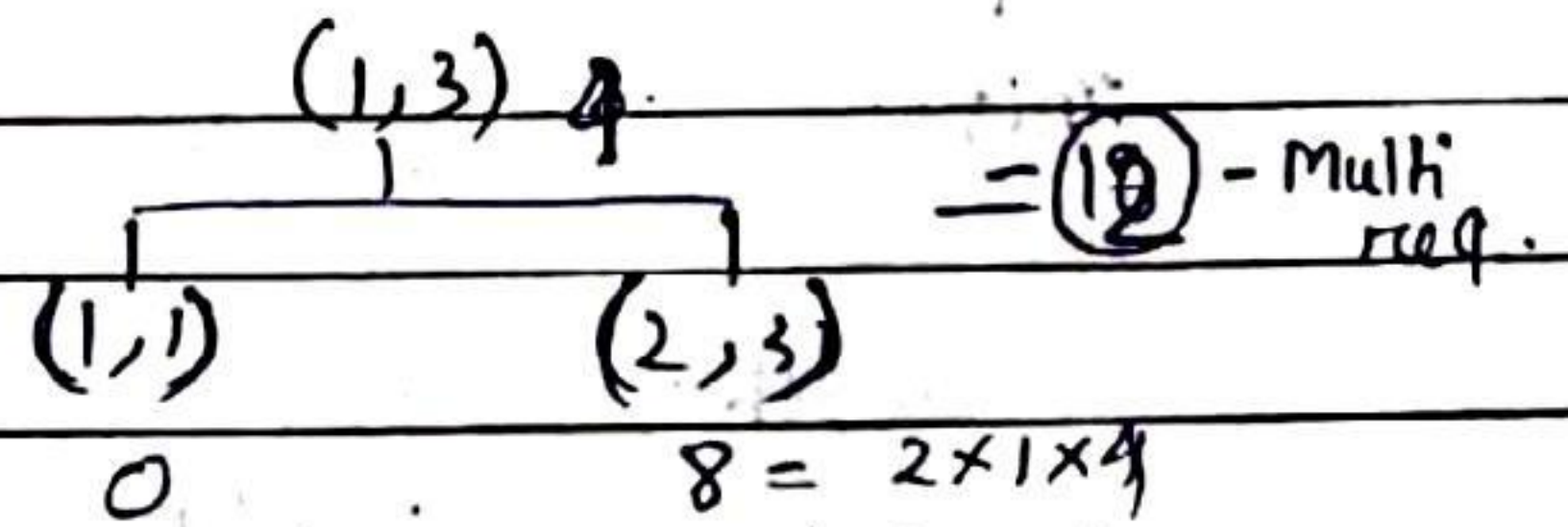
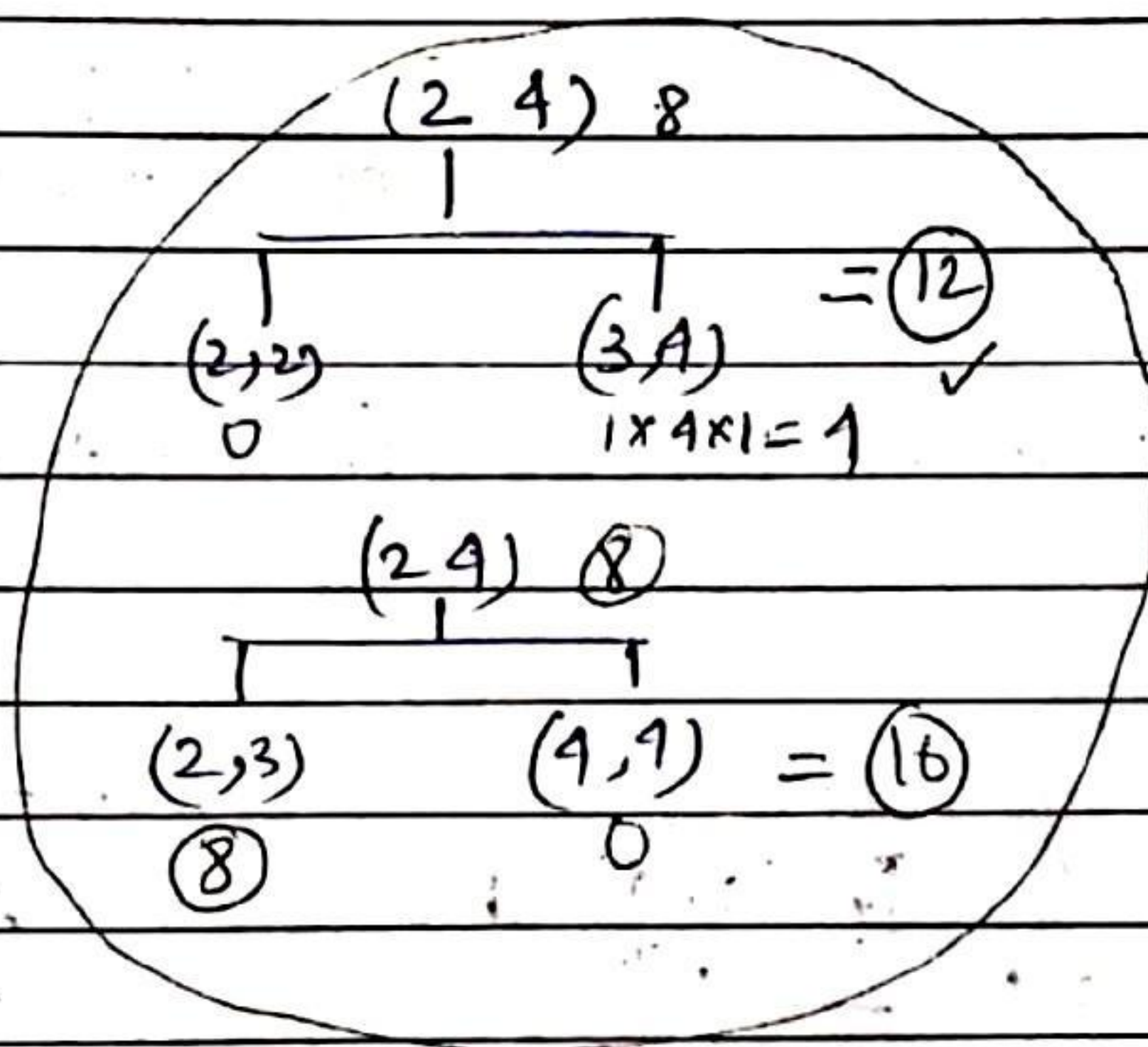
$$m[1, 4]$$

next page -

multiply '4'-matrix — $A_1 \quad A_2 \quad A_3 \quad A_4$
 $1 \times 2 \quad 2 \times 3 \quad 3 \times 4 \quad 4 \times 1$
 $P_0 P_1 P_2 P_3 P_4$

$(1,1)^0$	$(2,2)^0$	$(3,3)^0$	$(4,4)^0$
$(1,2)^2$	$(2,3)^8$	$(3,4)^4$	
$(1,3)^4$	$(2,4)^6$		
$(1,4)^7$			

$- O(n^2) O(n)$
 $\checkmark = O(n^3)$ — Time taken,

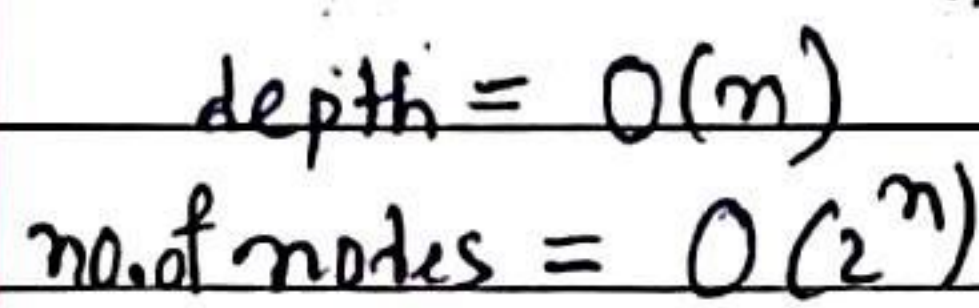


or

$$(2,4) = \min \begin{cases} (2,2) + (3,4) + P_1 P_2 P_4 = 0 + 12 + 2 = 14 \\ (2,3) + (4,4) + P_1 P_3 P_4 = 8 + 0 + 8 = 16 \end{cases}$$

$$(1,4) = \min \begin{cases} (1,1) + (2,4) + P_0 P_1 P_4 = 0 + 8 + 2 = 10 \\ (1,2) + (3,4) + P_0 P_2 P_4 = 2 + 12 + 1 = 15 \\ (1,3) + (4,4) + P_0 P_3 P_4 = 4 + 0 + 4 = 8 \end{cases}$$

Here, time complexity = $O(n^3)$
 and space complexity = $O(n^2)$.



- Dynamic programming bottom up implementation =

$$4 + 3 + 2 + 1$$
$$= \frac{n(n+1)}{2}$$

So, number of subproblems
= $O(n^2)$.

next page -

Lookup - Chain (m, p, i, j)

{

1. if $m[i, j] < \infty$

2. return $m[i, j]$

3. if $i = j$

4. $m[i, j] = 0$

5. else for $k = i$ to $j-1$

6. $q = \text{Lookup - Chain}(m, p, i, k) +$
 $\text{Lookup - Chain}(m, p, k+1, j) +$

$p_{i-1} p_k p_j$

7. if $q < m[i, j]$

8. $m[i, j] = q$

9. return $m[i, j]$

}

Time Complexity = $O(n^3)$.

Space Complexity = $O(n^2)$.

gate-2011

Question - (1)

given 4-Matrices M_1, M_2, M_3, M_4 and their dimensions are $(10 \times 100)(100 \times 20)(20 \times 5)(5 \times 80)$ respectively.

find minimum scalar multiplication.

→ $M_1 \quad M_2 \quad M_3 \quad M_4$
 $(10 \times 100) \quad (100 \times 20) \quad (20 \times 5) \quad (5 \times 80)$
 $p_0 \quad p_1 \quad p_1 \quad p_2 \quad p_2 \quad p_3 \quad p_3 \quad p_4$

20000
8 160000

	0	0	0	0
	11	2 2	3 3	4 4
1	20000	16000	8000	
2	15000	5000	39	
3	15000	2 4		
4	15000			

$$(1,3) = \begin{cases} (11) + (23) + p_0 p_1 p_3 = 15000 \\ (12) + (33) + p_0 p_2 p_3 = 21000 \end{cases}$$

$$(2,4) = \begin{cases} (2,2) + (34) + p_1 p_2 p_4 = 168000 \\ (2,3) + (44) + p_1 p_3 p_4 = 50000 \end{cases}$$

$$(1,4) = \begin{cases} (1,2) + (34) + p_0 p_2 p_4 = 160000 \\ (1,3) + (44) + p_0 p_3 p_4 = 15000 \end{cases}$$

MATRIX-Chain(P)

{
no of matrices.

A_1	A_2	A_3	A_4
1×2	2×1	1×4	4×1
1	2	2	1
1	4	4	1
P_0	P_1	P_2	P_3

P_0	P_1	P_2	P_3	P_4
-------	-------	-------	-------	-------

1. $n = P.length - 1$ 2. let $m[1 \dots n, 1 \dots n]$ and $s[1 \dots n-1, 2 \dots n]$ be new tables3. for $i = 1$ to n 4. $m[i, i] = 0$ 5. for $l = 2$ to n // l is the chain length.6. for $i = 1$ to $n - l + 1$ 7. $j = i + l - 1$ 8. $m[i, j] = \infty$ 9. for $k = i$ to $j - 1$ 10. $q = m[i, k] + m[k+1, j] + P_{i-1} P_k P_j$ ~~$T.C = O(n^2) O(n) = O(n^3)$~~ 11. if $q < m[i, j]$ 12. $m[i, j] = q$ 13. $s[i, j] = k$ 14. return m and s .

{

$l=1$	11	22	33	44
$l=2$	12	23	34	
$l=3$	13	24		
$l=4$	14			

• Top down dynamic programming memorization algorithm

MEMOIZED-MATRIX-CHAIN(P)

{

1. $n = P.length - 1$ 2. let $m[1 \dots n, 1 \dots n]$ be a new table3. for $i = 1$ to n 4. for $j = 1$ to n 5. $m[i, j] = \infty$ 6. return LOOKUP-CHAIN($m, P, 1, n$).

{

dimension array

$$(1, A) = \begin{cases} (11) + (24) + P_0 P_1 P_A = 130000 \\ \quad 20,000 \quad 8000 \quad 16,000 \\ (12) + (34) + P_0 P_2 P_A = 41000 \\ \quad 15000 \quad 0 \quad 4000 \\ (13) + (44) + P_0 P_3 P_A = 19000 \text{ (least cost)} \end{cases}$$

$$((M_1(M_2 M_3))M_4) = 19000 \text{ unit cost.}$$